

# HIGH-PERFORMANCE COMPUTING (HPC)

## System Environment & Practical Setup Guide

*Course Instructor: Prof. Abhijith Anandakrishnan  
School of Artificial Intelligence, Amrita Vishwa Vidyapeetham*

Welcome to the High-Performance Computing (HPC) practical sessions! This guide is designed to help you configure your personal laptops with the necessary tools to compile and run MPI, OpenMP, CUDA, and Cloud Computing workloads.

### Important Note on Operating Systems:

HPC tools and libraries are heavily optimized for Linux environments. While it is possible to use macOS or Windows, a Linux environment (specifically **Ubuntu 22.04 LTS or 24.04 LTS**) will provide the smoothest experience for this course.

## Phase 1: Operating System Setup

### Option A: Dual Booting Windows & Ubuntu (Recommended)

Dual booting allows you to keep Windows while dedicating a portion of your hard drive to Ubuntu, giving you native hardware access (critical for CUDA/GPU tasks).

- Create unallocated space:** In Windows, open "Disk Management". Right-click your main drive (usually C:) and select "Shrink Volume". Free up at least **50 GB** (100 GB recommended). Leave it as "Unallocated".
- Create a Bootable USB:** Download the [Ubuntu 24.04 LTS ISO](#). Use a tool like [Rufus](#) to flash the ISO onto an 8GB+ USB drive.
- BIOS / UEFI Quirks:** Before installing, restart your PC and enter the BIOS/UEFI settings (usually by pressing F2, F12, DEL, or ESC during startup).
  - Disable **Secure Boot**.
  - Ensure SATA Mode is set to **AHCI** (Not RAID/RST). *Warning: Changing this may require Windows Safe Mode boot first.*
- Disable Fast Startup:** In Windows, go to Control Panel > Power Options > "Choose what the power buttons do" > Uncheck "Turn on fast startup".

5. **Install:** Boot from the USB. Choose "Install Ubuntu". When prompted for installation type, select **"Install Ubuntu alongside Windows Boot Manager"**.

## Option B: Windows Subsystem for Linux (WSL 2) - The Ultimate Fallback

If dual-booting is too risky or complicated, **WSL 2** is the absolute best alternative. Do NOT use VirtualBox. VirtualBox cannot pass your physical GPU through to the virtual machine, meaning CUDA will not work. WSL 2 natively supports CUDA passthrough on Windows 11.

Open Windows PowerShell **as Administrator** and run:

```
wsl --install
```

This single command enables the required features and installs Ubuntu. Restart your computer when prompted. Once restarted, an Ubuntu terminal will open asking you to create a UNIX username and password.

### WSL & CUDA Note:

If you are using WSL 2, you must install the NVIDIA drivers on **Windows**, NOT inside Ubuntu. WSL will automatically detect the Windows GPU driver. Inside WSL, you only need to install the CUDA Toolkit.

## Phase 2: Base System & Developer Tools

Once you have booted into Ubuntu (or opened your WSL Ubuntu terminal), open the terminal ( **ctrl + Alt + T** ) and run the following commands sequentially.

### 1. Update and Upgrade Packages

```
sudo apt update && sudo apt upgrade -y
```

### 2. Install Build Essentials (C/C++ Compilers & Make)

This installs **gcc** , **g++** , and **make** , which are mandatory for compiling HPC code.

```
sudo apt install build-essential cmake pkg-config -y
```

### 3. Install Git & Basic Utilities

```
sudo apt install git wget curl htop nano vim -y
```

### 4. Set up Visual Studio Code

VS Code is highly recommended. To install it on native Ubuntu:

```
sudo snap install --classic code
```

*Note for WSL users:* Install VS Code on your Windows side, and install the "WSL" extension. You can then type `code .` inside your WSL terminal to open it.

**Recommended VS Code Extensions:** C/C++, CMake Tools, Makefile Tools, Remote - SSH.

## Phase 3: HPC Libraries (MPI, OpenMP, CUDA)

---

### 1. OpenMP Setup

OpenMP is typically bundled with the GCC compiler. However, to ensure all headers and libraries are present, install the development package:

```
sudo apt install libomp-dev -y
```

To compile OpenMP code: `gcc -fopenmp program.c -o program`

### 2. MPI (Message Passing Interface) Setup

We will use OpenMPI, one of the most widely used MPI implementations.

```
sudo apt install openmpi-bin libopenmpi-dev -y
```

Verify the installation by checking the compiler version:

```
mpicc --version  
mpirun --version
```

To compile MPI code: `mpicc program.c -o program`

### 3. CUDA Setup (For NVIDIA GPUs only)

**Warning:**

Only proceed with this if your laptop has a dedicated NVIDIA GPU.

**For Native Ubuntu Users:**

First, detect and install the recommended NVIDIA drivers:

```
sudo ubuntu-drivers autoinstall  
sudo reboot
```

After rebooting, install the NVIDIA CUDA Toolkit:

```
sudo apt install nvidia-cuda-toolkit -y
```

**For WSL 2 Users:**

Ensure you have the latest NVIDIA drivers installed on your Windows host. Then, inside WSL, install the toolkit (do NOT install driver packages inside WSL):

```
sudo apt install nvidia-cuda-toolkit -y
```

**Verify CUDA Installation:**

```
nvcc --version  
nvidia-smi
```

## Phase 4: Cloud Computing Preparation

For cloud computing modules (AWS/DigitalOcean), you will need SSH keys to securely access remote instances.

### Generate an SSH Key Pair

```
ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press Enter to accept the default file location. You can choose to add a passphrase for extra security. Your public key (which you will upload to the cloud provider) can be viewed using:

```
cat ~/.ssh/id_ed25519.pub
```

## Phase 5: Useful References & Links

---

- **WSL Setup Guide:** [Microsoft Official WSL Docs](#)
- **CUDA on WSL Guide:** [NVIDIA CUDA on WSL](#)
- **OpenMPI Documentation:** [Open MPI Docs](#)
- **OpenMP Tutorial:** [LLNL OpenMP Guide](#)

Happy Coding! See you in the practical sessions.